Documentação Técnica Integral: COMINDIR05

Implementação do Algoritmo RAFT

Projeto de Sistemas Distribuídos

12 de novembro de 2026

Conteúdo

1 Introdução e Visão Geral **2**

2 Resumo Integral da Implementação RAFT **2**

3 Estrutura dos Projetos e Ficheiros **3**
- 3.1 Projeto `raft04-no` . . . 3
- 3.2 Projeto `raft04-produtor` . . . 3
- 3.3 Projeto `raft04-processador` . . . 3

4 Compilação e Execução dos Projetos **3**
- 4.1 Compilação . . . 3
- 4.2 Ordem de Execucao . . . 3

5 RaftCheck e CausalCheck **3**
- 5.1 RaftCheck . . . 3
- 5.2 CausalCheck . . . 4

6 Cenarios de Execucao e Resultados Esperados **4**

7 Análise Individual de Todas as Classes **4**
- 7.1 Wire . . . 4
- 7.2 VoteRequest e VoteReply . . . 4
- 7.3 RaftState . . . 4
- 7.4 HeartbeatMonitor . . . 5
- 7.5 Batch . . . 5
- 7.6 RecordFile . . . 5
- 7.7 Producer . . . 6
- 7.8 AlarmProcessor . . . 6
- 7.9 PartitionNode . . . 6

8 Análise Detalhada de `tratarEntrada` e Registo de IGNORADO **7**

9 Fluxo Completo de Execução **7**

10 Conclusão **7**

1 Introdução e Visão Geral

O sistema implementa o algoritmo de consenso RAFT para resolver o problema de um líder fixo que, ao falhar, bloqueava indefinidamente a partição de dados. Para tal, introduz grupos de três nós idênticos por partição, capazes de eleger um novo líder democraticamente e gerir termos de liderança.

O projeto divide-se em três aplicações:

- **Produtor (No):** Lê sensores e publica leituras no tópico IPFS.
- **Nós de Partição (No):** Três réplicas que elegem um líder e replicam os dados.
- **Processador (No):** Ouve os lotes, funde os relógios e gera alarmes.

2 Resumo Integral da Implementação RAFT

A inicialização do sistema arranca com os nós a lerem os seus ficheiros de estado (se existirem). Cada nó inicia um `RaftState` como SEGUIDOR no termo gravado. Uma thread verifica a cada 200ms se o prazo de eleição expirou. Se expirar, o termo é incrementado, o nó vota em si próprio e passa a CANDIDATO, publicando um `PEDIDO_VOTO` (CORREÇÃO: underscore escapado).

Os restantes nós avaliam o pedido com base na restrição de voto (o candidato tem de ter o log mais atualizado). Se a maioria (2 de 3) votar a favor, o candidato torna-se LIDER e inicia a publicação de `LOTE`s (heartbeats) a cada 2 segundos.

Se um líder falhar, os seguidores deixam de receber os LOTEs. Após o timeout, é iniciada uma nova eleição com um termo superior. O antigo líder, ao voltar, verá o termo superior e demitir-se-á.

3 Estrutura dos Projetos e Ficheiros

3.1 Projeto `raft04-no`

Contém a lógica central. Ficheiros principais: `PartitionNode.java`, `RaftState.java`, `Batch.java`, `RecordFile.java`, `VoteRequest.java`, `VoteReply.java`, `SensorRecord.java`, `VectorClock.java`, `Wire.java`, `IpfsPubSub.java`, `Partitioner.java`.

3.2 Projeto `raft04-produtor`

Contém `Producer.java`, `HeartbeatMonitor.java` e as classes comuns (Batch, SensorRecord, etc.).

3.3 Projeto `raft04-processador`

Contém `AlarmProcessor.java`, `HeartbeatMonitor.java` e as classes comuns.

4 Compilação e Execução dos Projetos

4.1 Compilação

```
1         mkdir -p raft04-no/out raft04-produtor/out raft04-processador/out
2         javac -d raft04-no/out raft04-no/src/raft04/*.java
3         javac -d raft04-produtor/out raft04-produtor/src/raft04/*.java
4         javac -d raft04-processador/out raft04-processador/src/raft04/*.java
5
```

4.2 Ordem de Execução

1. **IPFS Daemon:** Executar `ipfs daemon`.
2. **Nós de Partição (9 processos):**

```
# Exemplo para particao 0, nos 1, 2 e 3
java -cp raft04-no/out raft04.PartitionNode 0 1 3
java -cp raft04-no/out raft04.PartitionNode 0 2 3
java -cp raft04-no/out raft04.PartitionNode 0 3 3
# Repetir para particao 1 e 2

```

3. **Processador:** `java -cp raft04-processador/out raft04.AlarmProcessor 3`
4. **Produtor:** `java -cp raft04-produtor/out raft04.Producer 3`

5 RaftCheck e CausalCheck

5.1 RaftCheck

Verifica a segurança da eleição lendo os ficheiros de logs gerados. Conta os termos com mais de um líder (deve ser 0), verifica se os termos voltaram atrás, calcula o tempo de indisponibilidade em falhas e conta as leituras perdidas (cruzando o `produtor.txt` com os ficheiros `no-*`). Também deteta os eventos de truncagem.

5.2 CausalCheck

Verifica a integridade dos relógios lógicos. Confirma se $V[k] = seq$ em todos os registos de todos os nós (garantindo que um novo líder não reiniciou o vetor a zero), e se o relógio de Lamport respeita a causalidade entre partições diferentes.

Aspeto	**RaftCheck**	**CausalCheck**
Objetivo	Validar segurança e disponibilidade RAFT	Validar causalidade e Lamport
Observa	Logs, eleições, hb, produtor	Ficheiros de registos (`no-*`)
Deteta	Dois líderes, leituras perdidas, truncagem	Quebras de V[k]=seq ou Lamport

6 Cenários de Execução e Resultados Esperados

Cenário	**Ação**	**Resultado Esperado**	**Verificação**
Arranque	Iniciar 9 nós	Líder eleito no termo 1, 0 votos divididos	RaftCheck: 1 termo
Falha do Líder	Matar o líder (kill -9)	Nova eleição, indisponibilidade de 6-12s	RaftCheck mede tempo
Partição Rede	Isolar o líder (ficheiro isolar)	Lado maioritário elege novo líder. Lado minoritário trunca dados ao voltar.	RaftCheck: TRUNCADO
Falha Maioria	Matar 2 nós	Sem líder, nó sobrevivente incrementa termo	RaftCheck: sem líder

7 Análise Individual de Todas as Classes

7.1 Wire

Identificação: Projeto No, Produtor e Processador.
Responsabilidade: Guardar constantes globais e gerar nomes de tópicos/portos.
Métodos e Atributos: `PERIODO_HB_MS` (2000ms), `ELEICAO_MIN/MAX_MS`, `isolated(k,i)` (cria um isolamento lógico de rede).

7.2 VoteRequest e VoteReply

Identificação: Projeto No.
Responsabilidade: DTOs para as mensagens de eleição.
Métodos: `encode()` e `decode(String)` convertem os dados (termo, candidato, ultimoSeq) para texto.

7.3 RaftState

Identificação: Projeto No.
Responsabilidade: O cérebro lógico da eleição (termos, votos, prazos).
Atributos: `term` e `votedFor` (persistentes); `role`, `leader`, `votes` (voláteis).

Método `persist()`: Grava no disco de forma atómica para garantir que um nó não vota duas vezes.

```
1       private void persist() {
2           try {
3               Path tmp = stateFile.resolveSibling(stateFile.getFileName() + ".
    tmp");
4               Files.writeString(tmp, term + ";" + votedFor + "\n",
    StandardCharsets.UTF_8);
5               Files.move(tmp, stateFile, StandardCopyOption.REPLACE_EXISTING,
    StandardCopyOption.ATOMIC_MOVE);
6           } catch (IOException e) {
7               throw new UncheckedIOException(e);
8           }
9       }
10
```

Método `onVoteRequest()`: Aplica a restrição de voto. Só vota se o candidato estiver atualizado.

```
1       public synchronized Vote onVoteRequest(long cTerm, int candidate, long
    cLastSeq, long cLastTerm,
2       long myLastSeq, long myLastTerm) {
3           if (cTerm > term) adoptTerm(cTerm);
4           boolean upToDate = cLastTerm > myLastTerm
5           || (cLastTerm == myLastTerm && cLastSeq >= myLastSeq);
6           boolean granted = cTerm == term
7           && (votedFor == 0 || votedFor == candidate)
8           && upToDate;
9           if (granted) {
10              votedFor = candidate;
11              persist();
12              resetTimer();
13          }
14          return new Vote(term, granted);
15      }
16
```

7.4 HeartbeatMonitor

Identificação: Projetos Produtor e Processador.
Responsabilidade: Detetar falhas do líder através da ausência de LOTEs.
Método `onHeartbeat()`: Ignora termos obsoletos e regista `NOVO_LIDER` se o termo subir. O `check()` dispara `SUSPEITA` após 6000ms de silêncio.

7.5 Batch

Identificação: Projeto No, Produtor e Processador.
Responsabilidade: Transportar o cabeçalho do LOTE (`term`, `leader`, `lastSeq`, `lastTerm`) e os registos.

7.6 RecordFile

Identificação: Projeto No.
Responsabilidade: Escrever no disco.
Método `rewrite()`: Usado na truncagem para substituir o ficheiro completo atomicamente.

```
1         public synchronized void rewrite(List<SensorRecord> registos) throws
    IOException {
2             StringBuilder sb = new StringBuilder();
3             for (SensorRecord r : registos) {
4                 sb.append(r.toLine()).append(System.lineSeparator());
5             }
6             Path tmp = path.resolveSibling(path.getFileName() + ".tmp");
7             Files.writeString(tmp, sb.toString(), StandardCharsets.UTF_8);
8             Files.move(tmp, path, StandardCopyOption.REPLACE_EXISTING,
    StandardCopyOption.ATOMIC_MOVE);
9         }
10
```

7.7 Producer

Identificação: Projeto Produtor.
Responsabilidade: Ler do teclado e publicar `IN`.
Método `publicar()`: Publica no IPFS e só regista no `produtor.txt` se a publicação tiver sucesso.

7.8 AlarmProcessor

Identificação: Projeto Processador.
Responsabilidade: Consumir LOTEs de todas as partições e gerar alarmes.
Método `receber()`: Filtra termos antigos (`batch.term < maiorTermo[k]`). Usa a deduplicação `seq:termo`. Funde os relógios de Lamport e Vetor antes de publicar o alarme.

7.9 PartitionNode

Identificação: Projeto No.
Responsabilidade: Orquestrar as threads de rede, ficheiros e delegar decisões ao `RaftState`.

Método `tornarLider()`: Inicializa `seq`, Lamport e Vetor a partir do último registo, garantindo a continuidade causal.

```
1         private void tornarLider() {
2             if (raft.termIfLeader() == -1) return;
3             seq = ultimoSeq();
```

```
            if (registos.isEmpty()) {
                lamport = 0;
                vector = new VectorClock(n);
            } else {
                SensorRecord ultimo = registos.get(registos.size() - 1);
                lamport = ultimo.lamport;
                vector = ultimo.vector.copy();
            }
            hb = 0;
            pendentes = new ArrayList<>();
            // inicia o agendamento a cada PERIODO_HB_MS...
        }

```

Método `executarRecuperacao()`: Lida com lacunas e divergências. Em divergência, usa o método `inicioBloco()` para encontrar o último termo, pede por TCP ao líder, e trunca o seu ficheiro.

```
        if (d <= ultimoSeq()) {
            List<SensorRecord> cauda = new ArrayList<>(registos.subList((int) d - 1, registos.size()));
            // guarda a cauda para o RaftCheck...
            registos.subList((int) d - 1, registos.size()).clear();
            ficheiro.rewrite(registos);
            evento("TRUNCADO;p" + k + ";de=" + d + ";descartados=" + cauda.size() + ...);
        }

```

8 Análise Detalhada de `tratarEntrada` e Registo de IGNORADO

O método `tratarEntrada` é invocado no `PartitionNode` pela thread que subscreve o tópico `topicIn`. Ele valida o sensor e a partição e, então, entra na secção crítica.

O que acontece se o nó não for líder:

```
        synchronized (this) {
            long t = raft.termIfLeader();
            if (t == -1) {
                evento("IGNORADO;p" + k + ";IN " + sensor + " " + temp + " (sem lideranca)");
                return;
            }
            // ... processamento normal do lider
        }

```

O que é o evento IGNORADO: A função `evento()` escreve no ficheiro `eventos-p\{k\}-n\{i\}.txt` de forma síncrona (usando `Files.writeString` com `APPEND`).

Impacto no SSD (Degradação):

- **Frequência:** Numa rajada de 1000 leituras, os 2 nós seguidores recebem os 1000 INs. Como não são líderes, escrevem 1000 linhas de `IGNORADO` em disco cada um (2000 escritas).
- **Desgaste:** Cada escrita abre o ficheiro, aloca espaço e fecha. Isto não só degrada a performance da JVM (I/O bloqueante), como submete o SSD a um ciclo de escrita massivo.
- **Mitigação no código:** O código atual não tem buffer. A mitigação ideal seria usar `BufferedWriter` com flush de 1 em 1 segundo, ou um rate-limiter para só escrever um log `IGNORADO` a cada X segundos, evitando a morte do SSD num cenário de alta carga.

9 Fluxo Completo de Execução

10 Conclusão

A implementação do RAFT no COMINDIR05 introduz uma solução robusta para falhas do líder através de termos e quóruns. O sistema garante a segurança (nunca há dois líderes num termo) mas prioriza a consistência sobre a disponibilidade (CP) em caso de partições de rede, sacrificando leituras antigas (truncagem) para manter a integridade.

As ferramentas `RaftCheck` e `CausalCheck` provam matematicamente o sucesso destas propriedades. A principal limitação identificada é a degradação de performance/SSD causada pelo logging síncrono excessivo de eventos `IGNORADO`, que em ambiente de produção exigiria técnicas de buffering.

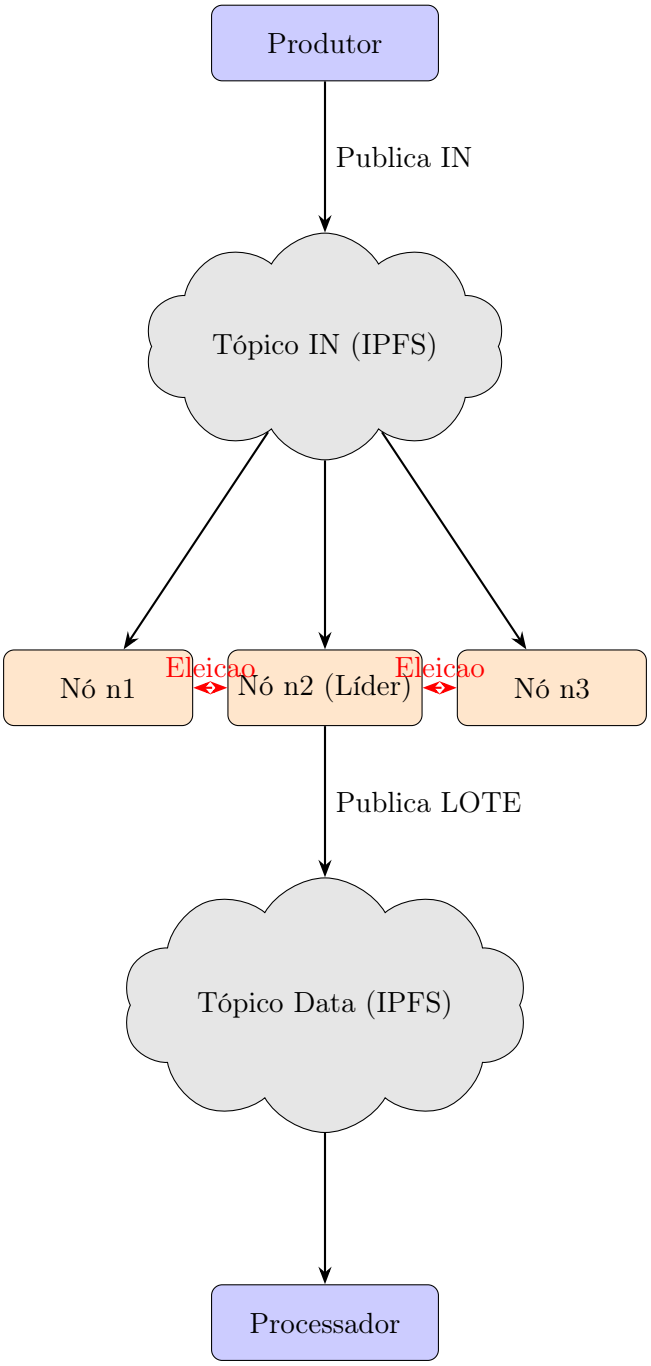


Figura 1: Arquitetura de Comunicação do Sistema

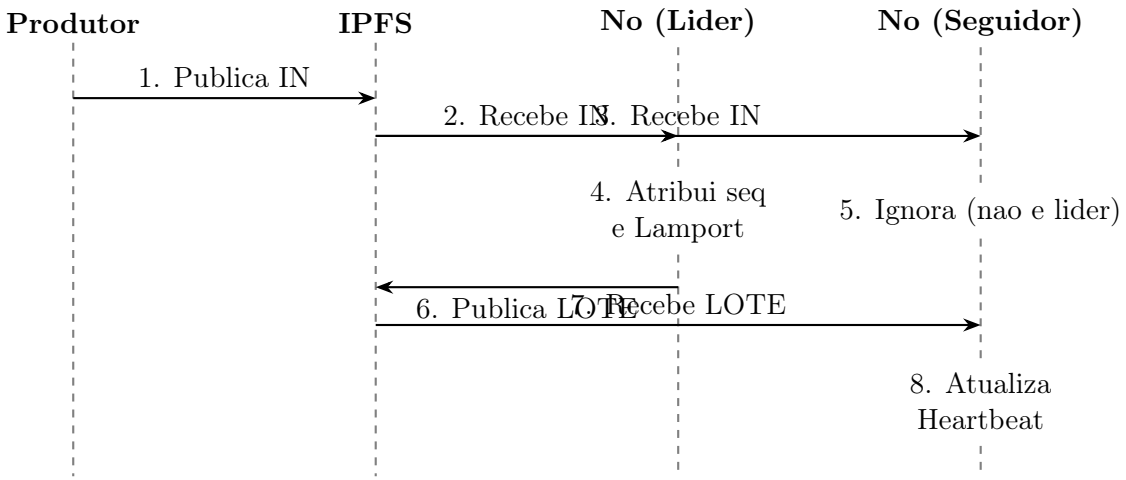


Figura 2: Fluxo de Execução de uma Leitura